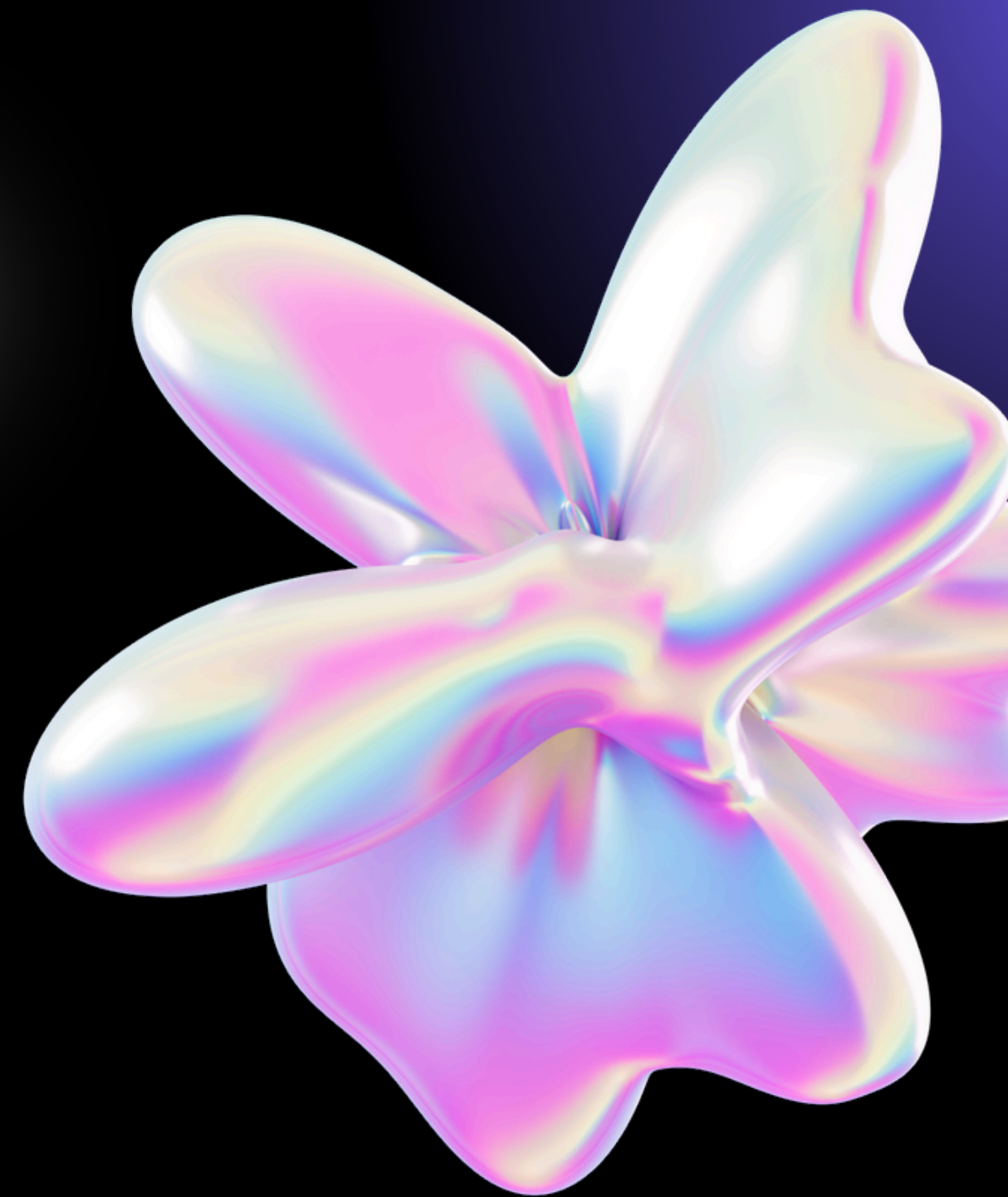


COE- LOTTO

กองสลาก PSU



Introduction

ในยุคที่การเข้าถึงอินเทอร์เน็ตสามารถทำได้ง่ายเพียงแค่ปลายนิ้ว การจะทำกิจกรรมใดๆนั้นลื่นง่ายขึ้นเป็นอย่างมาก ซึ่งหนึ่งเรื่องที่เราเห็นได้ชัดจากการเข้าถึงอินเทอร์เน็ตที่ง่ายขึ้นคือ การซื้อสินค้าออนไลน์

กลุ่มของเราได้เล็งเห็นถึงเรื่องนี้ จึงได้พัฒนาเว็บไซต์แอปพลิเคชัน ซึ่งเป็นเว็บไซต์เกี่ยวกับการจำหน่ายและตรวจสอบราคาสินค้าแบ่งรัฐบาลออนไลน์ โดยที่ผู้ซื้อสามารถซื้อสินค้าแบ่งรัฐบาลได้ทุกที่ทุกเวลา โดยไม่ต้องเดินทางไปหาซื้อที่แผงขาย หรือในช่วงที่มีเวลาจำกัด เช่น เมื่อมีเวลาว่างแค่ช่วงสั้น ๆ สามารถทำการซื้อได้ทันทีผ่านอินเทอร์เน็ต



Techniques

01 Git

02 Web development

03 Docker

04 Kubernetes

05 Load Balancer

06 Makefile

07 Snapshot

08 Backups

09 Security

10 Multiuser

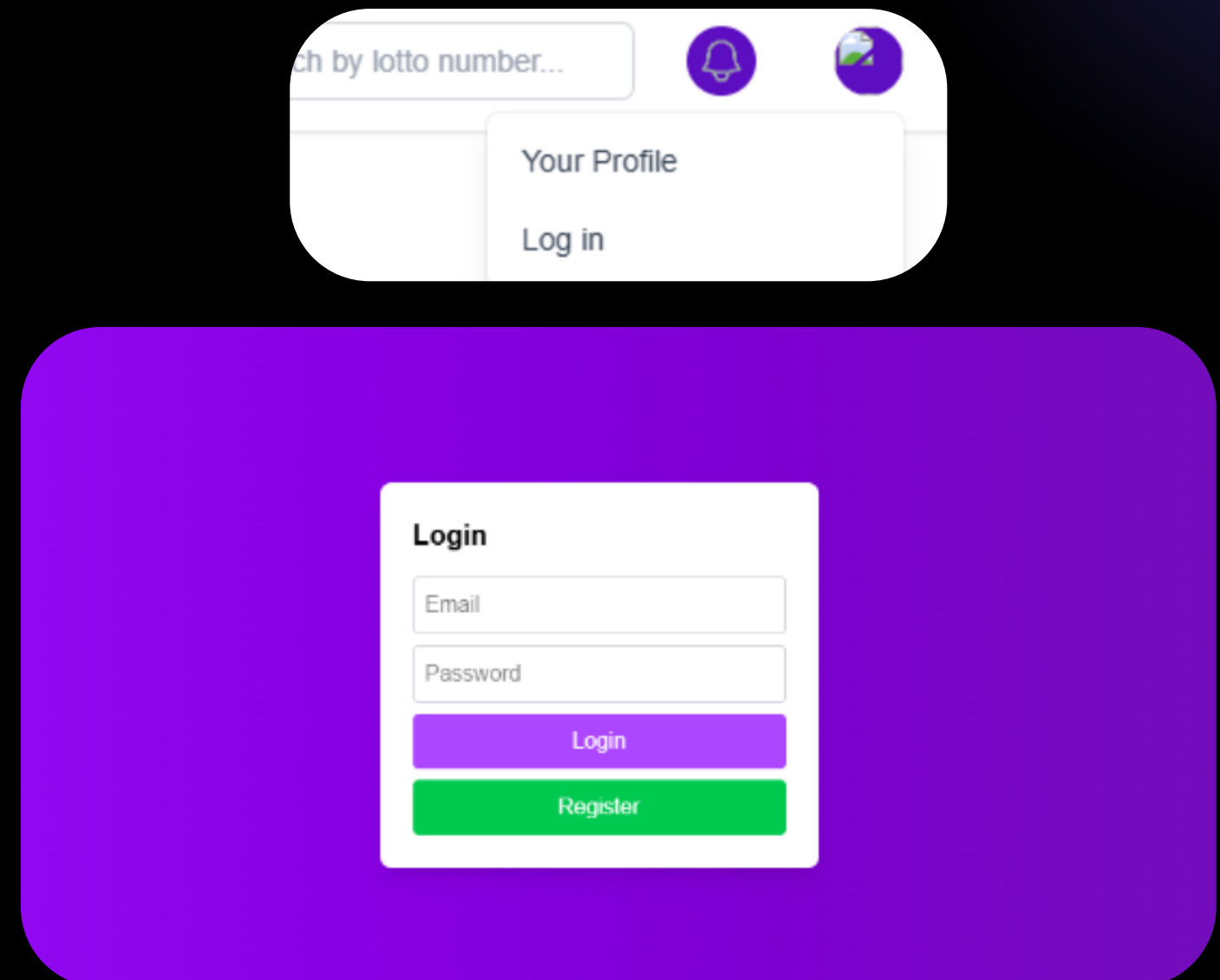
11 Concurrency

วิธีการใช้งาน

01 เข้าใช้งานเว็บไซต์ที่
URL: <https://coe-lotto.coedesigns.com/dashboards>

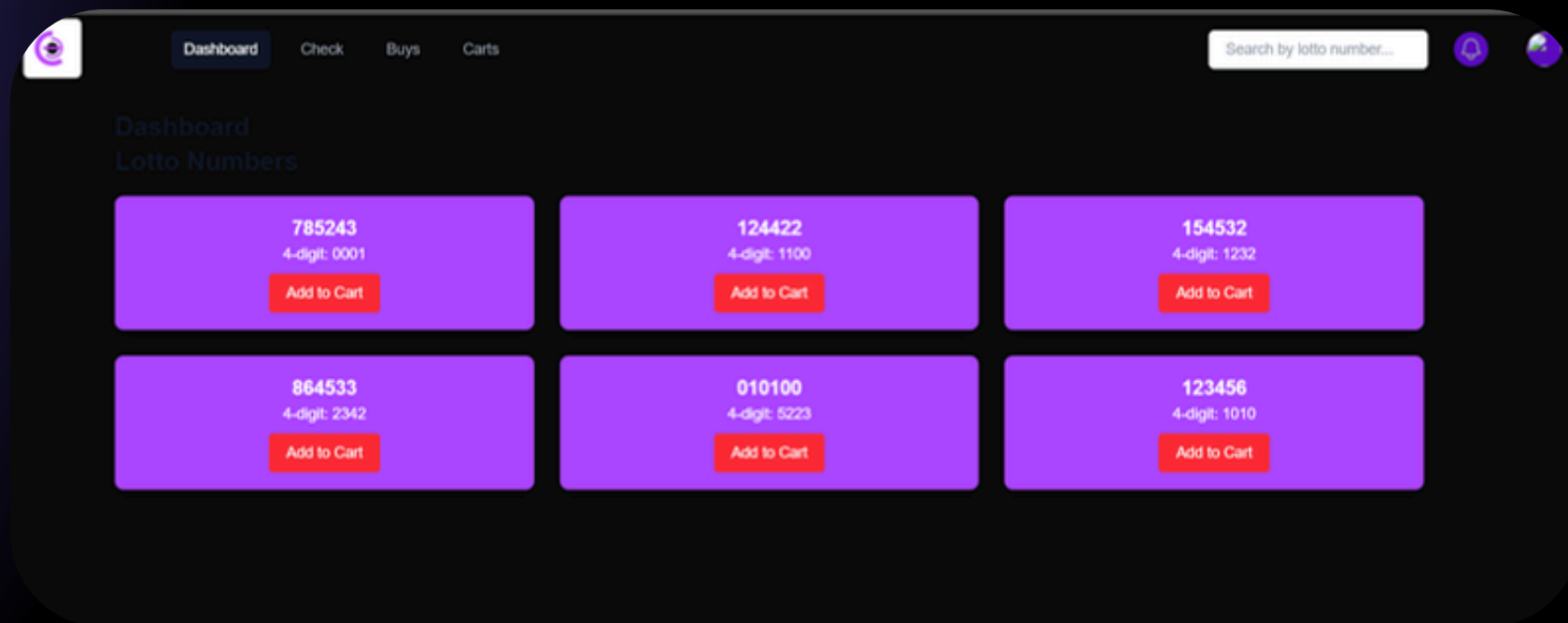


02 ทำการ Register หรือ Login ก่อนใช้งาน

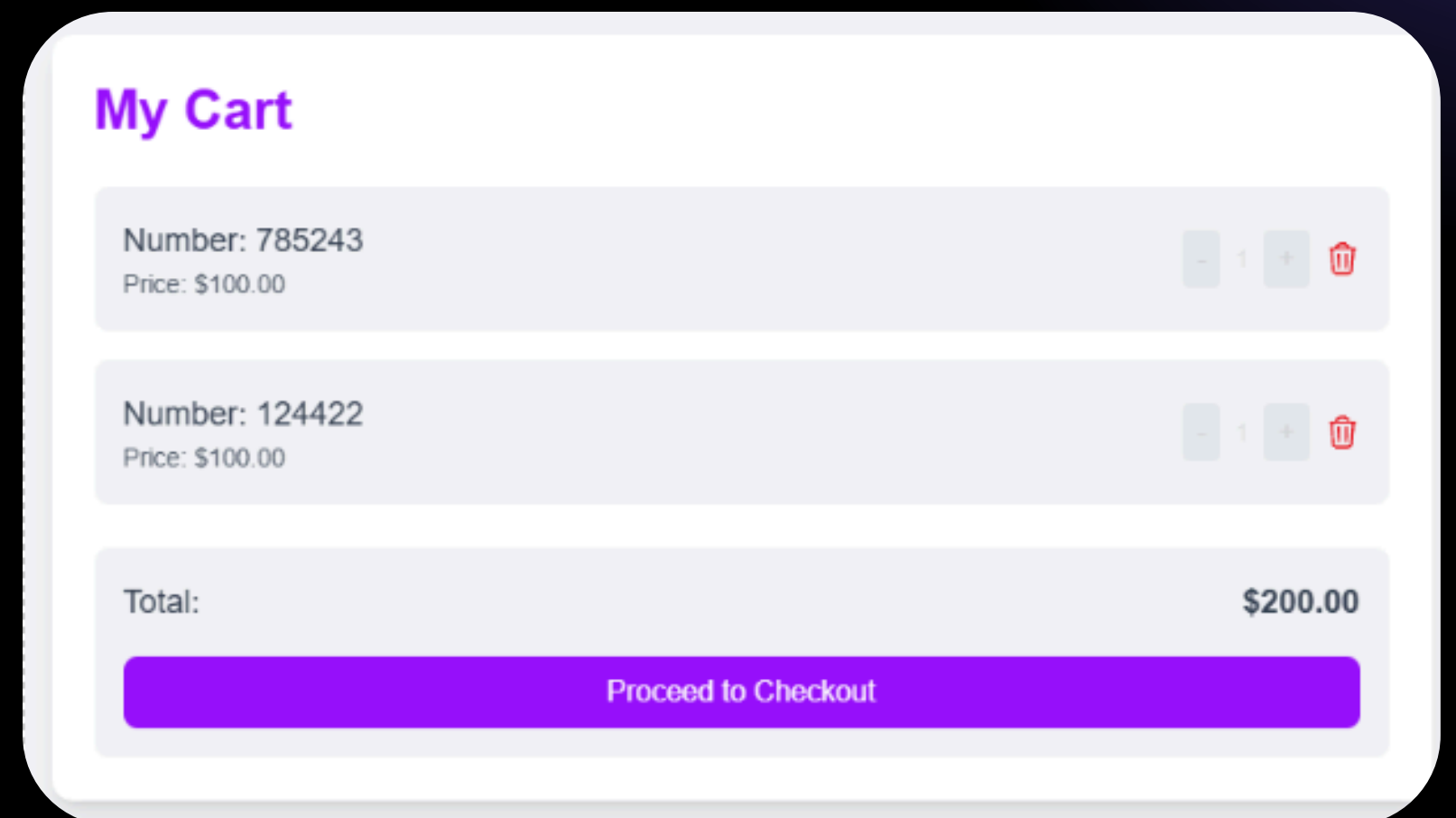


วิธีการใช้งาน

03 เมื่อLoginแล้วจะเข้ามาหน้า Dashboards อัตโนมัติ



04 ทำการRegister หรือ Login ก่อนใช้งาน



Technique

1. GIT

การควบคุมเวอร์ชัน

Git ช่วยในการติดตามการเปลี่ยนแปลงทั้งหมดของโค้ดและไฟล์ในโปรเจกต์ ทำให้สามารถย้อนกลับไปยังเวอร์ชันก่อนหน้าได้หากมีปัญหาเกิดขึ้น

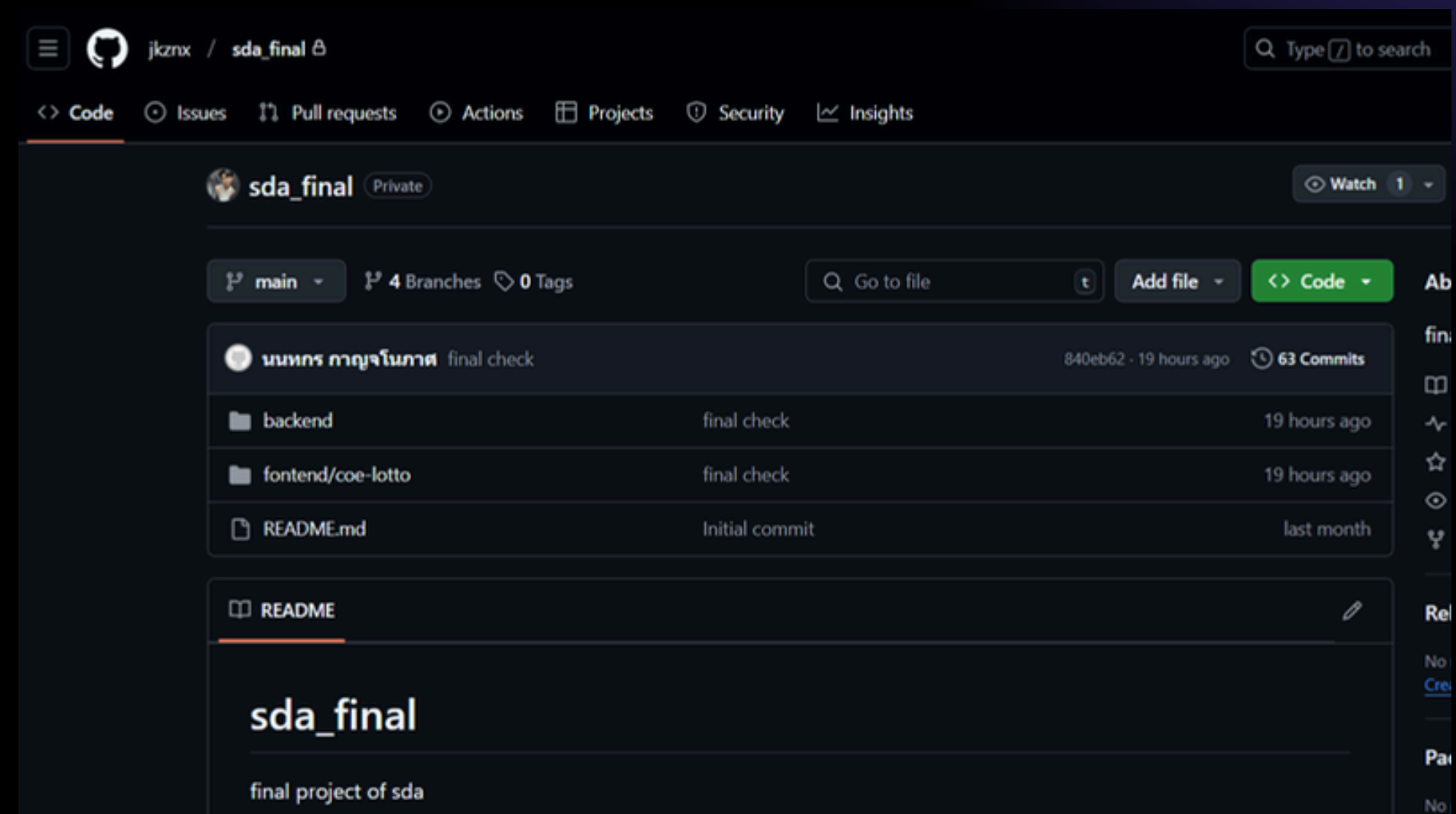
การทำงานร่วมกัน

GitHub ทำให้การทำงานร่วมกันในทีมง่ายขึ้น โดยสามารถแชร์โค้ดกับทีม, ส่ง pull request เพื่อเสนอการเปลี่ยนแปลง, และทำ review โค้ดของผู้อื่น

ความปลอดภัย

โค้ดของคุณจะถูกเก็บใน repository บน GitHub ทำให้ปลอดภัยจากการสูญหายและสามารถเข้าถึงได้จากที่ใดก็ได้ที่มีอินเทอร์เน็ต

Git เป็นระบบการควบคุมเวอร์ชัน (Version Control System: VCS) ที่ใช้กันอย่างแพร่หลาย โดยเฉพาะในวงการพัฒนาซอฟต์แวร์ ซึ่งใน project นี้ จะถูกใช้ผ่าน Github



Technique

2. WEB DEVELOPMENT

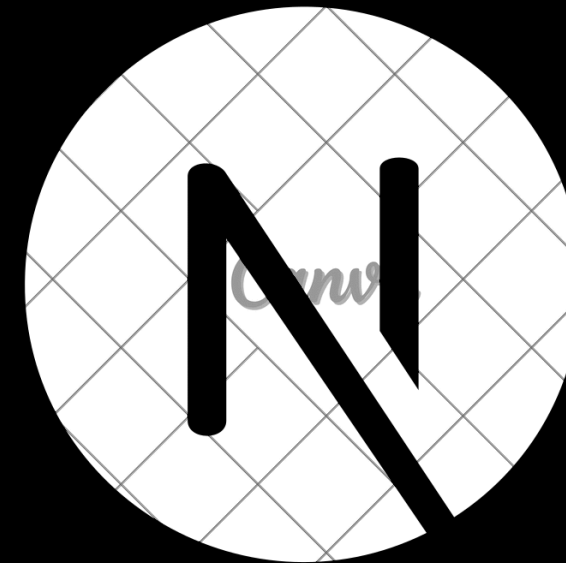
ในการพัฒนาเว็บไซต์ จะแบ่งออกเป็น ส่วน Frontend และ Backend

จัดการ BACKEND ด้วย STRAPI

การใช้ Strapi เป็น database ที่ใช้เก็บข้อมูลหมายเลขลอตเตอรี่, ข้อมูลผู้ใช้งาน, ผลรางวัลที่ออก

จัดการ FRONTEND ด้วย NEXT.JS

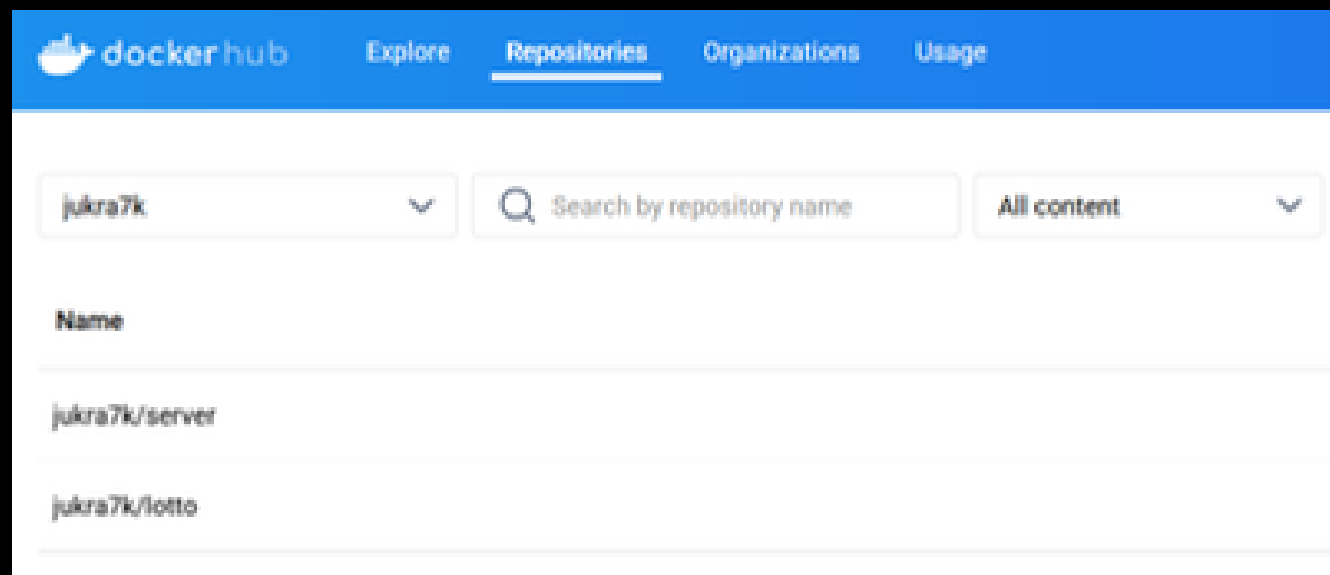
ซึ่งจะควบคุมในการ render ฝั่ง server และการ routing ไปยังหน้าต่างๆ และจัดการการใช้ css ของหน้าเว็บ จาก Tailwind



Technique

3. DOCKER

Docker ทำให้สามารถรวม dependencies ต่าง ๆ ที่จำเป็นสำหรับ frontend (เช่น Node.js, Nginx, ฯลฯ) ลงใน container เดียวกัน ทำให้ไม่ต้องกังวลเรื่อง version conflict หรือการติดตั้ง dependencies ในเครื่องแต่ละเครื่อง



```
Dockerfile > ...
1  FROM node:18
2
3  WORKDIR /usr/src/app
4
5  COPY package*.json ./
6
7  RUN npm install
8
9  COPY . .
10
11 EXPOSE 3000
12
13 CMD ["npm", "run", "dev"]
14
```

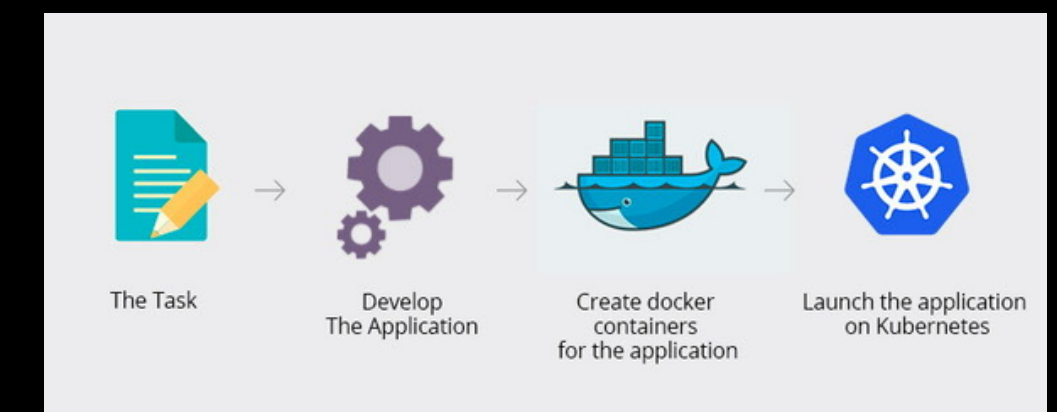
Technique

4. KUBERNETES

Kubernetes มีบทบาทสำคัญในการ deploy เว็บแอปพลิเคชันโดยให้แพลตฟอร์มที่แข็งแกร่งสำหรับการจัดการ containerized applications (แอปพลิเคชันที่อยู่ในรูปแบบคอนเทนเนอร์) โดยอัตโนมัติ ไม่ว่าจะเป็นการ deploy, การขยายขนาด (scaling), และการจัดการอื่น ๆ

```
PS C:\Users\ACER\Desktop\Projects\final-version\sda_final\deploy> kubectl create secret tls test-ingress-tls --cert=tls.crt --key=tls.key
secret/test-ingress-tls created
PS C:\Users\ACER\Desktop\Projects\final-version\sda_final\deploy> kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.yaml
namespace/cert-manager created
customresourcedefinition.apiextensions.k8s.io/certificaterequests.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/certificates.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/challenges.acme.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/clusterissuers.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/issuers.cert-manager.io created
customresourcedefinition.apiextensions.k8s.io/orders.acme.cert-manager.io created
serviceaccount/cert-manager-cainjector created
serviceaccount/cert-manager created
serviceaccount/cert-manager-webhook created
clusterrole.rbac.authorization.k8s.io/cert-manager-cainjector created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-issuers created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-clusterissuers created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-certificates created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-orders created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-challenges created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-ingress-shim created
clusterrole.rbac.authorization.k8s.io/cert-manager-cluster-view created
clusterrole.rbac.authorization.k8s.io/cert-manager-view created
clusterrole.rbac.authorization.k8s.io/cert-manager-edit created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-approve:cert-manager-io created
clusterrole.rbac.authorization.k8s.io/cert-manager-controller-certificatesigningrequests created
clusterrole.rbac.authorization.k8s.io/cert-manager-webhook:subjectaccessreviews created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-cainjector created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-issuers created
clusterrolebinding.rbac.authorization.k8s.io/cert-manager-controller-clusterissuers created
```

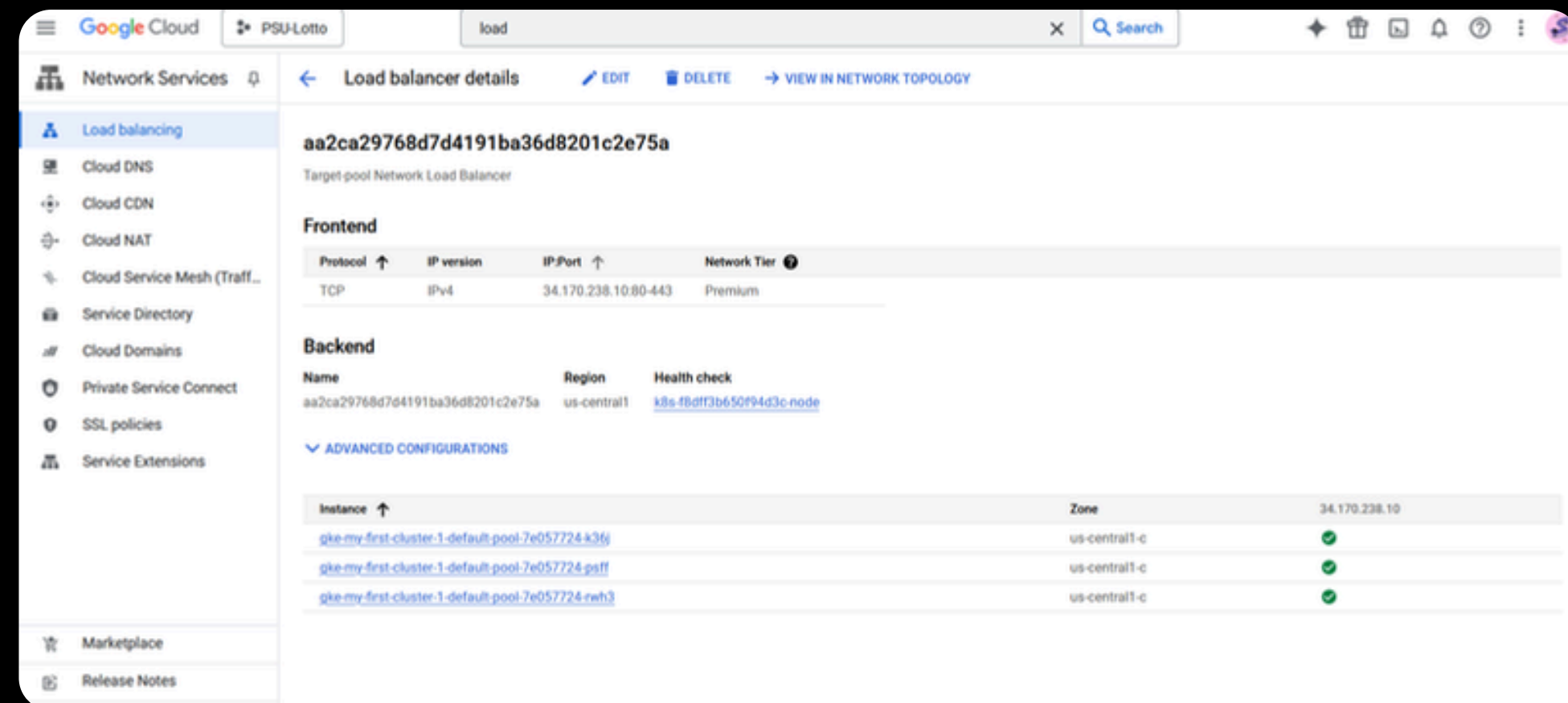
1. การจัดการคอนเทนเนอร์ (Container Orchestration) โดยอัตโนมัติ ทำให้เว็บแอปพลิเคชันทำงานได้อย่างสม่ำเสมอในสภาพแวดล้อมต่าง ๆ
2. การขยายขนาด (Scaling) โดยอัตโนมัติตามปริมาณ traffic และการใช้ทรัพยากร เพื่อให้ประสิทธิภาพและการใช้ต้นทุนเป็นไปอย่างเหมาะสม
3. การกระจายโหลด (Load Balancing) เพื่อกระจาย traffic ไปยัง instance ของเว็บแอปพลิเคชันอย่างเท่าเทียมกัน ช่วยเพิ่มความเร็วและความรวดเร็วในการตอบสนอง
4. การซ่อมแซมตัวเอง (Self-Healing) สามารถ restart คอนเทนเนอร์ที่ล้มเหลว, แทนที่คอนเทนเนอร์ใหม่, หรือย้ายไปยัง node อื่นเพื่อให้ระบบกลับมาทำงานตามที่กำหนด
5. การอัปเดตและย้อนกลับ (Rolling Updates and Rollbacks) ทำให้สามารถ deploy เวอร์ชันใหม่ของเว็บแอปพลิเคชันได้โดยไม่ทำให้ระบบหยุดทำงาน หากมีปัญหาเกิดขึ้นก็สามารถย้อนกลับไปเวอร์ชันก่อนหน้าได้ง่าย



Technique

5. LOADBALANCER

ใช้ในการกระจายการร้องขอ (requests) หรือการทำงาน (load) ไปยังเซิร์ฟเวอร์หลายๆ ตัวเพื่อให้การทำงานของระบบมีความเสถียรและมีประสิทธิภาพสูงสุด



The screenshot shows the Google Cloud console interface for a Network Load Balancer. The left sidebar lists various Network Services, with 'Load balancing' selected. The main content area displays the 'Load balancer details' for a specific balancer with ID 'aa2ca29768d7d4191ba36d8201c2e75a'. It identifies the balancer as a 'Target pool Network Load Balancer' and shows its 'Frontend' configuration: TCP protocol, IPv4, IP port 34.170.238.10:80-443, and Premium network tier. The 'Backend' section lists a single backend instance with name 'aa2ca29768d7d4191ba36d8201c2e75a', region 'us-central1', and a health check URL. Below this, the 'ADVANCED CONFIGURATIONS' section shows a table of backend instances, all in the 'us-central1-c' zone, with IP addresses and green status indicators.

Protocol	IP version	IP Port	Network Tier
TCP	IPv4	34.170.238.10:80-443	Premium

Name	Region	Health check
aa2ca29768d7d4191ba36d8201c2e75a	us-central1	k8s-fbdf3b650f94d3c-node

Instance	Zone	IP Address	Status
gke-my-first-cluster-1-default-pool-7e057724-k36j	us-central1-c	34.170.238.10	✓
gke-my-first-cluster-1-default-pool-7e057724-gsff	us-central1-c		✓
gke-my-first-cluster-1-default-pool-7e057724-rwh3	us-central1-c		✓

Technique

6. MAKEFILE

Makefile ช่วยให้การจัดการ build ต่าง ๆ ของโปรเจกต์มีความชัดเจน และเป็นระเบียบ ซึ่งช่วยให้การทำงานของโปรเจกต์มีประสิทธิภาพมากขึ้น

```
# Project and Cluster Configuration
PROJECT_ID = cloud-project-448820
REGION = asia-southeast1
CLUSTER = autopilot-strap1
SERVICE_ACCOUNT = cloud-sql-proxy@$(psu-lotto).iam.gserviceaccount.com

# Cloud SQL Instance
DB_INSTANCE = postgresql

# Kubernetes Deployment and Namespace
DEPLOYMENT = strapi-deploy
NAMESPACE = strapi
HPA = strapi-hpa
VPA = strapi-vpa
INGRESS = test-ingress # From your previous ingress.yml
SERVICE = strapi-deploy-service

# Files
DEPLOYMENT_FILE = deployment.yaml
HPA_FILE = HPA.yaml
VPA_FILE = VPA.yaml
INGRESS_FILE = ingress.yml

# Default target
.PHONY: help
help:
    @echo "Available commands:"
    @echo "  make start      - Start the Strapi server and PostgreSQL"
    @echo "  make stop       - Stop the Strapi server and PostgreSQL"
    @echo "  make apply      - Apply all Kubernetes resources"
    @echo "  make delete     - Delete all Kubernetes resources"
    @echo "  make status     - Check the status of resources"
    @echo "  make clean      - Clean up all resources"
```

```
# =====
# START SERVER
# =====
.PHONY: start
start: start-postgres apply-deployment apply-ingress

start-postgres:
    @echo "Starting PostgreSQL instance..."
    gcloud sql instances patch $(DB_INSTANCE) --activation-policy ALWAYS --project=$(PROJECT_ID) --quiet

apply-deployment:
    @echo "Applying deployment configuration..."
    gcloud container clusters get-credentials $(CLUSTER) --region $(REGION) --project $(PROJECT_ID)
    kubectl apply -f $(DEPLOYMENT_FILE) -n $(NAMESPACE)
    kubectl apply -f $(HPA_FILE) -n $(NAMESPACE)
    kubectl apply -f $(VPA_FILE) -n $(NAMESPACE)
    kubectl patch service $(SERVICE) -n $(NAMESPACE) --type json -p '[{"op": "replace", "path": "/spec/selector", "value": {"app": "strapi"}}]'
    @echo "Deployment applied successfully."

apply-ingress:
    @echo "Applying ingress configuration..."
    kubectl apply -f $(INGRESS_FILE) -n $(NAMESPACE)
    @echo "Ingress applied successfully."

# =====
# STOP SERVER
# =====
.PHONY: stop
stop: stop-postgres delete-strap1

stop-postgres:
    @echo "Stopping PostgreSQL instance..."
    gcloud sql instances patch $(DB_INSTANCE) --activation-policy NEVER --project=$(PROJECT_ID) --quiet
```

Technique

7. SNAPSHOT

การทำสำเนาของเว็บไซต์หรือเซิร์ฟเวอร์ที่ทำงานอยู่ในขณะนั้น เพื่อให้สามารถกลับไปใช้ข้อมูลหรือการตั้งค่าที่มีอยู่ในช่วงเวลานั้นได้ในอนาคต

Disks

CREATE DISKREFRESHDELETE

SHOW INFO PANEL

Filter Enter property name or value								
☐	Status	Name ↓	Type	Size	Architecture	Zone(s)	In use by	Snapshot schedule
☐	✓	instance-20250217-144641	Balanced persistent disk	50 GB	x86/64	asia-southeast1-a	coe-lotto	None
☐	✓	gke-my-first-cluster-1-default-pool-7e057724-rwh3	Balanced persistent disk	32 GB	x86/64	us-central1-c	gke-my-first-cluster-1-	None
☐	✓	gke-my-first-cluster-1-default-pool-7e057724-psff	Balanced persistent disk	32 GB	x86/64	us-central1-c	gke-my-first-cluster-1-	None
☐	✓	gke-my-first-cluster-1-default-pool-7e057724-k36j	Balanced persistent disk	32 GB	x86/64	us-central1-c	gke-my-first-cluster-1-	None
☐	✓	disk-20250306-204203	Balanced persistent disk	20 GB	—	us-central1-a		schedule-1

Snapshot schedules

DELETESCHEDULEEDIT SCHEDULE

schedule-1	
Region	us-central1
Schedule frequency (UTC)	Every Sunday, starts between 1:00 AM and 2:00 AM
Autodelete snapshots after	7 days
Deletion rule	When deleting the disk that uses this schedule, keep snapshots
Snapshot location	us (United States)
Enabled Volume Copy Shadow Service (VSS)	No
Snapshot labels	None
In use by	disk-20250306-204203
Creation time	Mar 7, 2025, 3:43:08 AM UTC+07:00

Technique

8. BACK UP

เก็บBackupของKubernetes เสมือนเก็บตัวโคลนของคลัสเตอร์ หากมีปัญหาก็กทำการBackupกลับได้

Backup for GKE

[+ Create a backup plan](#)

[+ Create a restore plan](#)

Configure backup and restore operations to protect workloads in your Kubernetes clusters. Backup for GKE is a separate service from GKE with independent certifications and accreditation. [Learn more](#)

Configure log-based alerts to get notified on backup failures or paused backup plans. [Learn more](#)

Backup plans

Restore plans

Backups

Restores

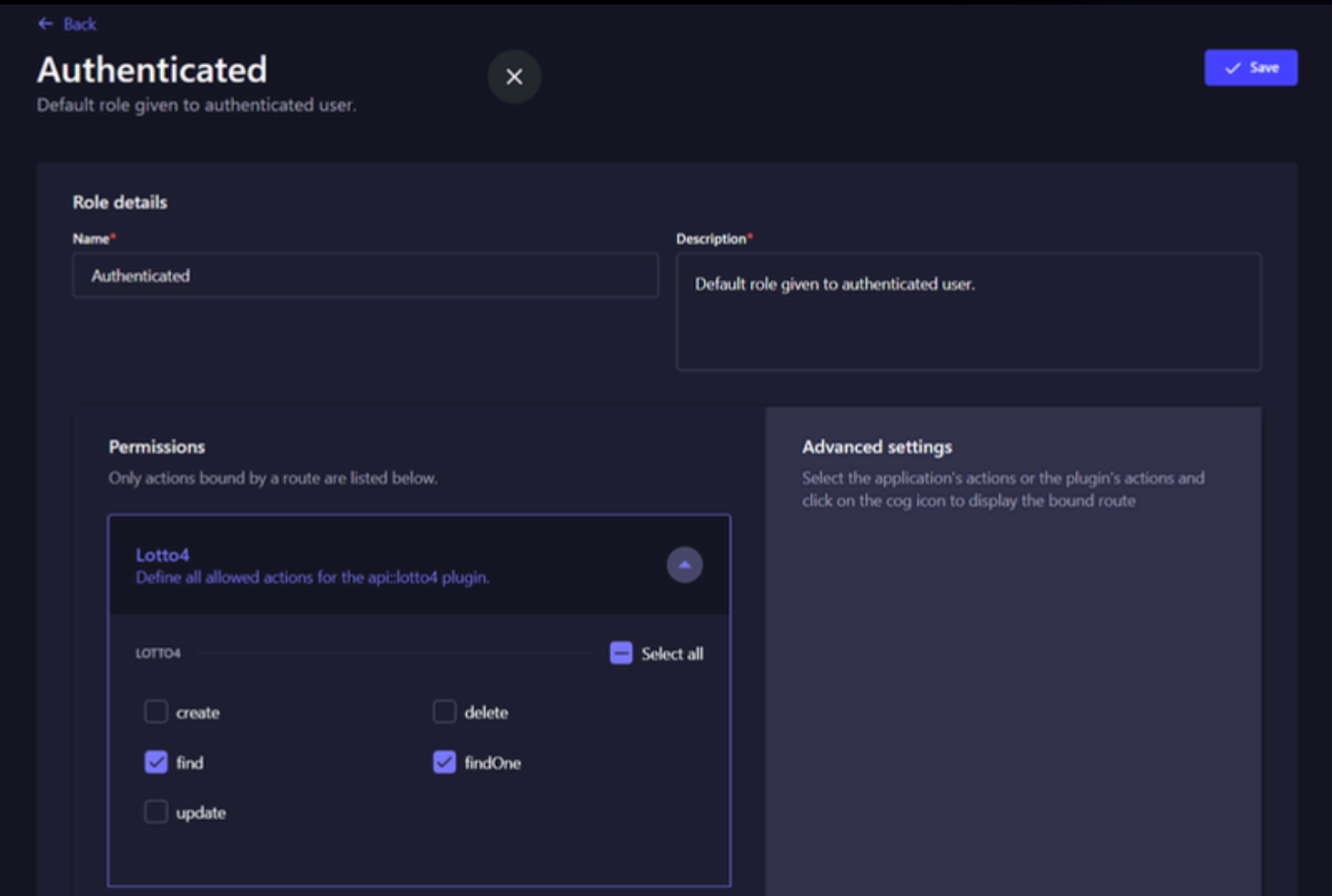
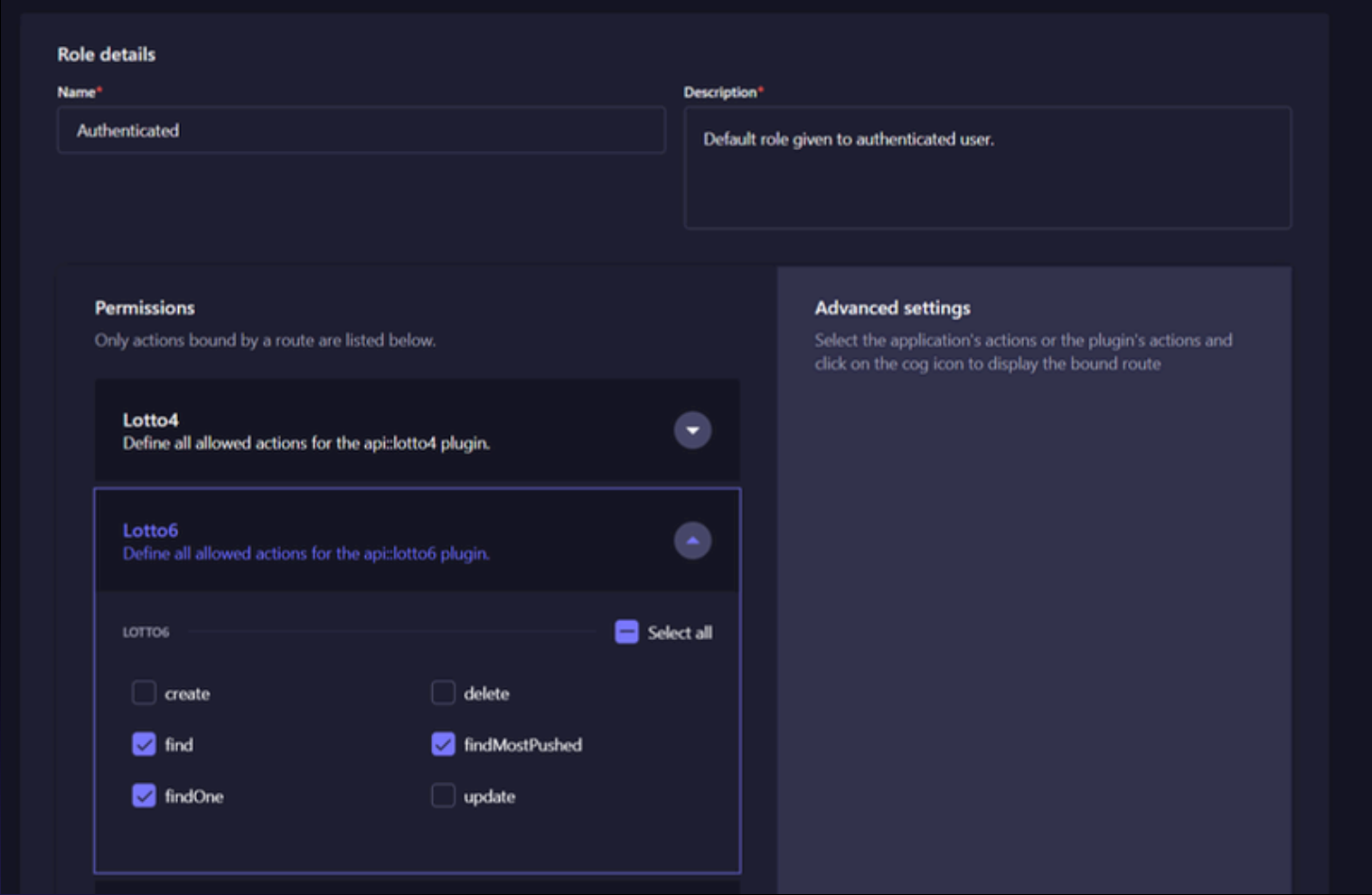
Use backup plans to configure and administer backups.

Cluster / Backup plan	Plan status	RPO risk	Backup location	Pods	Last backup time	Next backup	
my-first-cluster-1			us-central1-c				+ Backup plan
my-first-cluster-1-backup-1		Low	us-central1	6	March 7, 2025 at 3:54:47 AM GMT+7	March 22, 2025 at 3:44:47 AM GMT+7	More actions

Technique

9. SECURITY

- การจัดการสิทธิ์การเข้าถึง (Role-based Access Control)
- สร้างระบบการจัดการสิทธิ์ของผู้ใช้ เช่น ผู้ดูแล (admin), ผู้ใช้ทั่วไป (user), หรือผู้ใช้ที่มีสิทธิ์พิเศษ.
- กำหนดสิทธิ์การเข้าถึงข้อมูลหรือฟีเจอร์ต่างๆ โดยขึ้นอยู่กับบทบาทของผู้ใช้แต่ละคน.



Technique

10. MULTI USER

การสมัครสมาชิก (Sign-up) และการเข้าสู่ระบบ (Login)

- สร้างระบบที่รองรับการลงทะเบียนผู้ใช้ใหม่ และการเข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่าน หรือวิธีการอื่น ๆ เช่น การเข้าสู่ระบบผ่าน Google หรือ Facebook.
- ใช้เทคนิคการเข้ารหัสรหัสผ่าน (เช่น bcrypt) เพื่อป้องกันข้อมูลผู้ใช้
- การแบ่งปันข้อมูลระหว่างผู้ใช้



Technique

10. CONCURRENCY

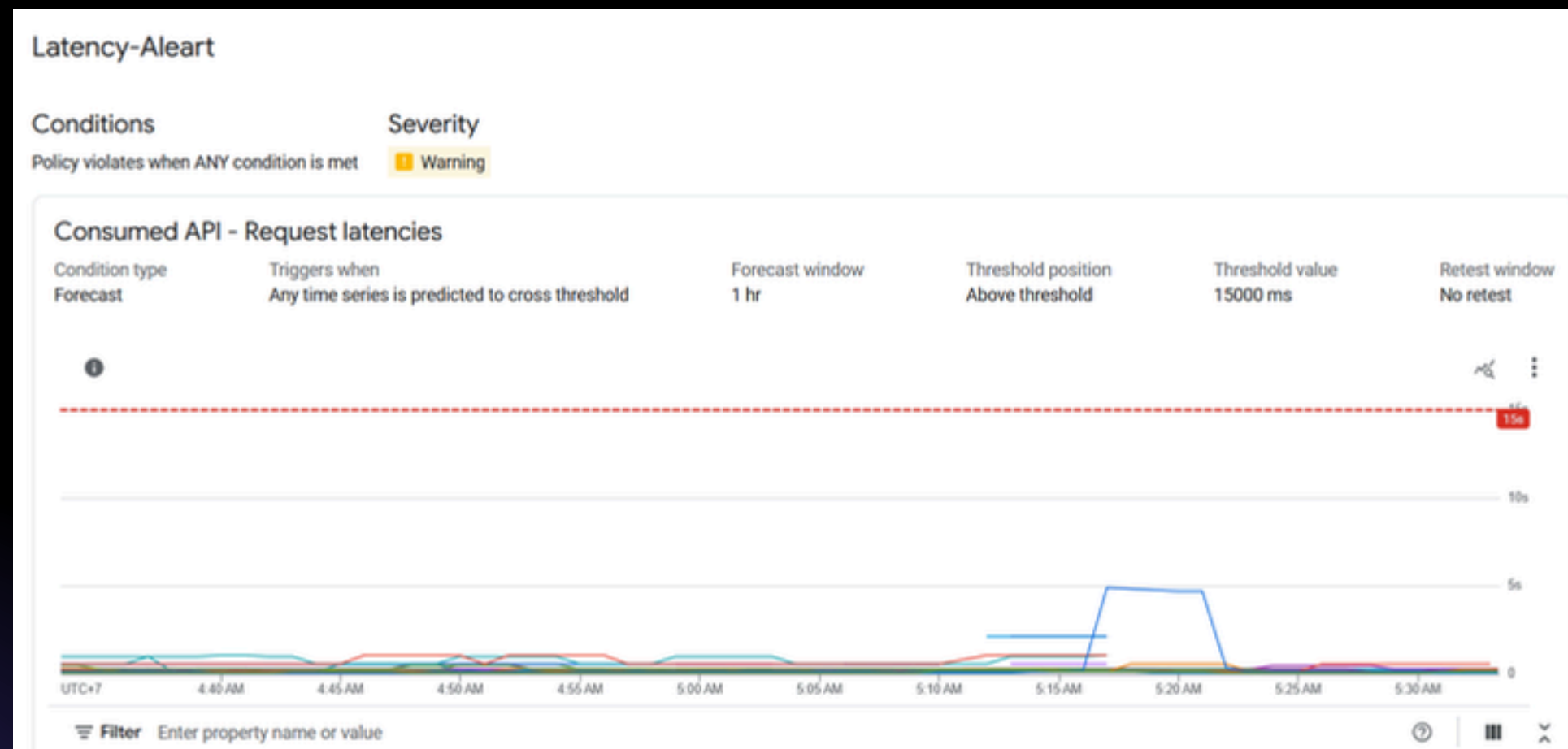
Strapi ซึ่งเป็น Headless CMS ที่สร้างขึ้นด้วย Node.js มีการรองรับการทำงานหลายๆ อย่างพร้อมกัน (concurrent requests) โดยอาศัยพีเจอร์ต่างๆ ที่ Node.js และฐานข้อมูลที่ใช้งาน (เช่น MongoDB, PostgreSQL, MySQL) รองรับ



Technique

11.MONITORING

เป็นการตั้งค่าแจ้งเตือนเมื่อมีความล่าช้าในการขอ Request เกิน 15 วินาที



Member



นายณฐนนท์ ณ พัทลุง
6610110077



นายจักรชัย ปล้องไหม
6610110428



นายณนทกร กาญจโนภาศ
6610110137

Thank you for your attention